Trabajo Final Integrador – Programación 2

Aplicación Java con Relación 1→1 Unidireccional + DAO + MySQL

Integrantes del Grupo y Roles

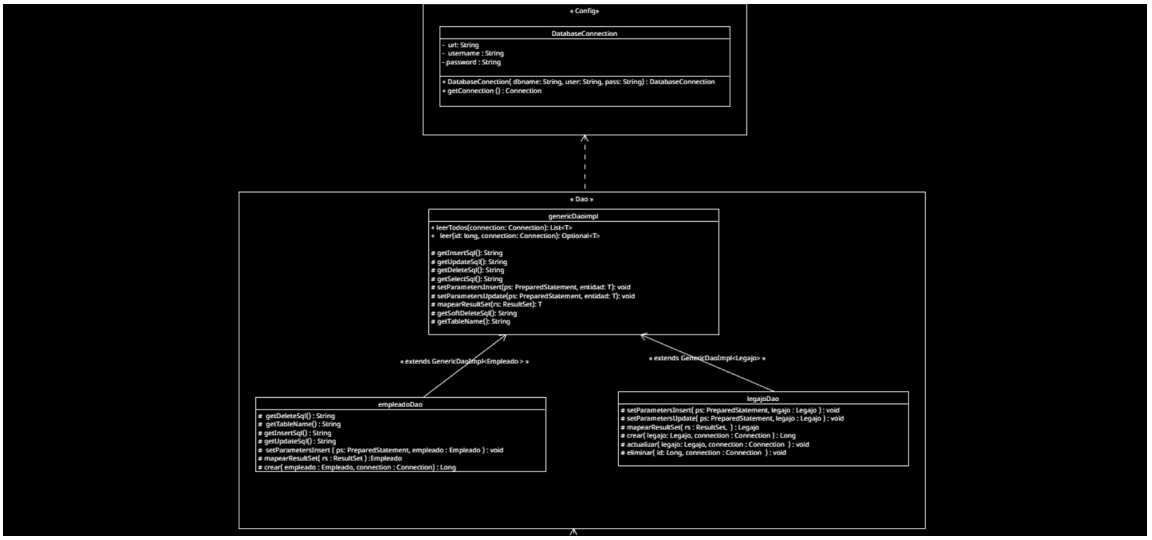
Nombre	**Rol y tareas realizadas**
Elizondo, Iñaki	Desarrollo del diseño UML y modelado de clases. Implementación inicial de la estructura del proyecto y paquetes.
Ducoli, Maximiliano David	Implementación de la capaDAOy configuración de conexión a la base de datos (DatabaseConnection). Desarrollo de scripts SQL para creación de tablas y carga de datos de prueba.
Farías, Gustavo	Desarrollo de la capaService con manejo transaccional (commit/rollback), validaciones de negocio y orquestación de operaciones. Implementación del menú de consola (AppMenu) con manejo robusto de errores y búsquedas funcionales.

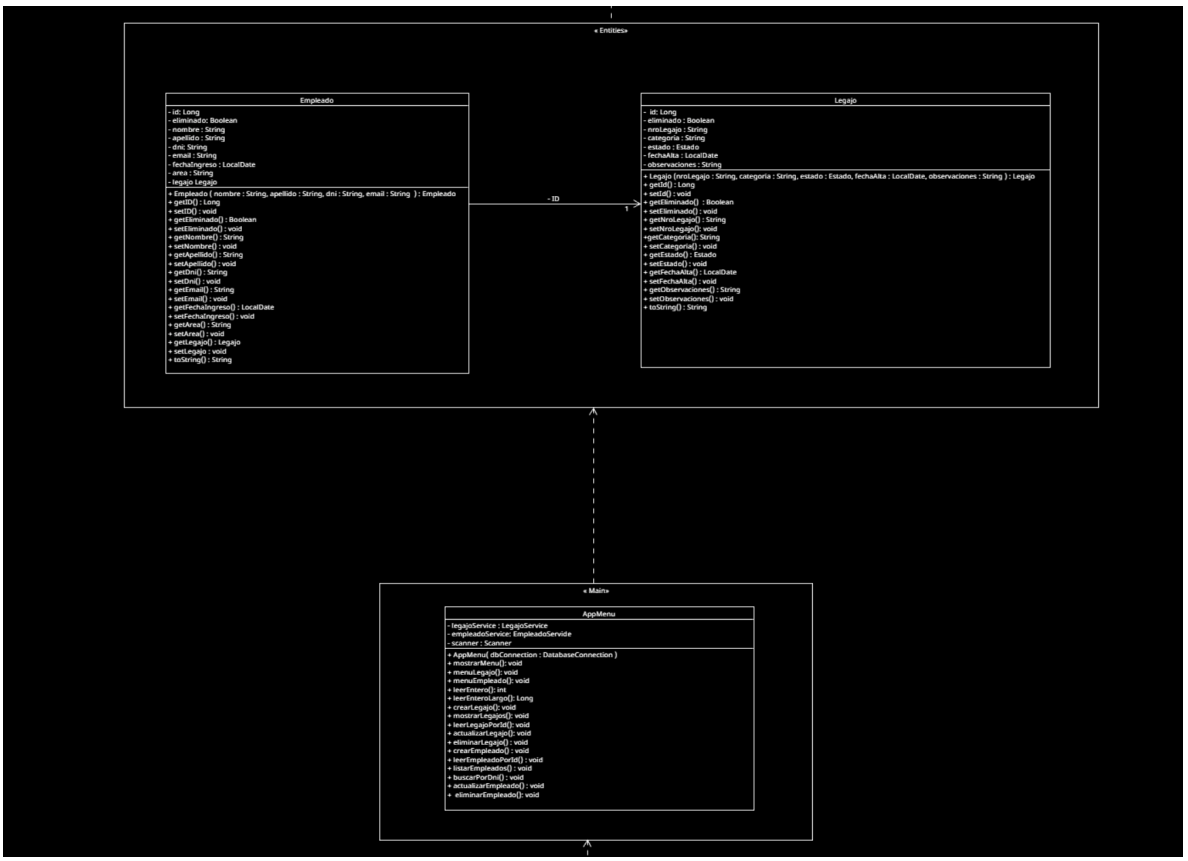
Equipo conformado por 3 integrantes. El cuarto miembro no contestó a los mensajes para las reuniones y el tiempo ya no nos iba a alcanzar para seguir buscando otro integrante.

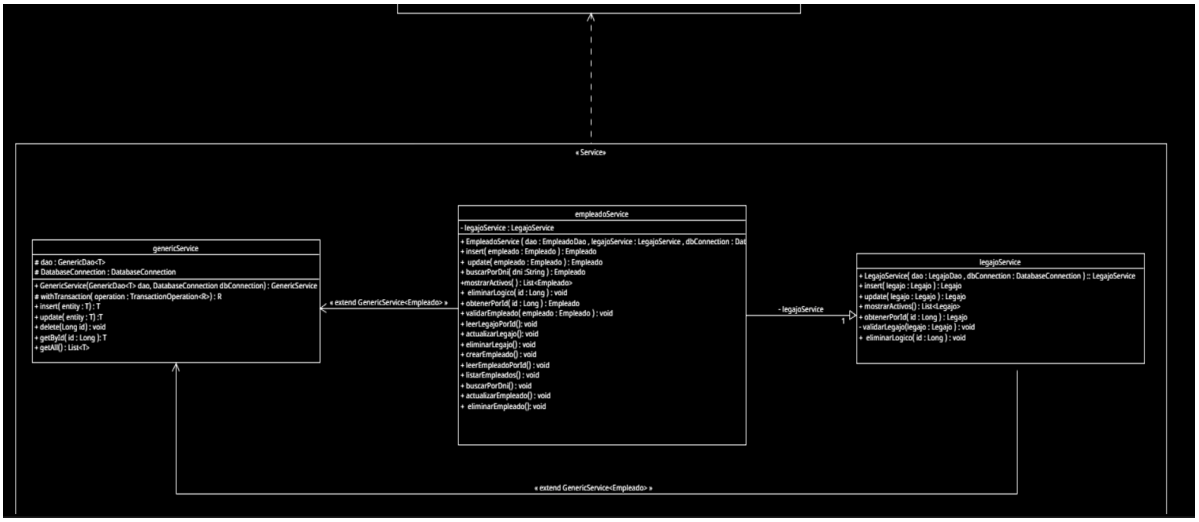
- ➔ **Repositorio GitHub:** https://github.com/maxiducoli/TrabajoPracticoIntegrador_Programacion_II
- ➔ **Repositorio Drive:** https://drive.google.com/file/d/1rngVOi8TYAfpe1QRqiI7M3AtdjIfZAZb/view

---

1. UML y clases comunes


« Config»
DatabaseConnection
- url: String
- username : String
- password : String
+ DatabaseConection(dbname: String, user: String, pass: String) : DatabaseConnection
+ getConnection () : Connection
« Dao »
genericDaoImpl
+ leerTodos(connection: Connection): List<T>
+ leer(id: long, connection: Connection): Optional<T>
# getInsertSql(): String
# getUpdateSql(): String
# getDeleteSql(): String
# getSelectSql(): String
# setParametersInsert(ps: PreparedStatement, entidad: T): void
# setParametersUpdate(ps: PreparedStatement, entidad: T): void
# mapearResultSet(rs: ResultSet): T
# getSoftDeleteSql(): String
# getTableName(): String
« extends GenericDaoImpl<Empleado > »
« extends GenericDaoImpl<Legajo> »
empleadosDao
getDeleteSql() : String
getTableName() : String
getInsertSql() : String
getUpdateSql() : String
setParametersInsert (ps: PreparedStatement, empleado : Empleado) : void
mapearResultSet(rs : ResultSet) :Empleado
crear(empleado : Empleado, connection : Connection) : Long
legajoDao
setParametersInsert(ps: PreparedStatement, legajo : Legajo) : void
setParametersUpdate(ps: PreparedStatement, legajo : Legajo) : void
mapearResultSet(rs : ResultSet,) : Legajo
crear(legajo: Legajo, connection : Connection) : Long
actualizar(legajo: Legajo, connection : Connection) : void
eliminar(id: Long, connection : Connection) : void


« Entities»
Empleado
- id: Long
- eliminado: Boolean
- nombre : String
- apellido : String
- dni: String
- email : String
- fechaIngreso : LocalDate
- area : String
- legajo Legajo
+ Empleado (nombre : String, apellido : String, dni : String, email : String) : Empleado
+ getID() : Long
+ setID() : void
+ getEliminado() : Boolean
+ setEliminado() : void
+ getNombre() : String
+ setNombre() : void
+ getApellido() : String
+ setApellido() : void
+ getDni() : String
+ setDni() : void
+ getEmail() : String
+ setEmail() : void
+ getFechaIngreso() : LocalDate
+ setFechaIngreso() : void
+ getArea() : String
+ setArea() : void
+ getLegajo() : Legajo
+ setLegajo : void
+ toString() : String
- ID
1
Legajo
- id: Long
- eliminado : Boolean
- nroLegajo : String
- categoria : String
- estado : Estado
- fechaAlta : LocalDate
- observaciones : String
+ Legajo (nroLegajo : String, categoria : String, estado : Estado, fechaAlta : LocalDate, observaciones : String) : Legajo
+ getId() : Long
+ setId() : void
+ getEliminado() : Boolean
+ setEliminado() : void
+ getNroLegajo() : String
+ setNroLegajo() : void
+getCategoria() : String
+ setCategoria() : void
+ getEstado() : Estado
+ setEstado() : void
+ getFechaAlta() : LocalDate
+ setFechaAlta() : void
+ getObservaciones() : String
+ setObservaciones() : void
+ toString() : String
« Main»
AppMenu
- legajosService : LegajoService
- empleadosService: EmpleadoServide
- scanner : Scanner
+ AppMenu(dbConnection : DatabaseConnection)
+ mostrarMenu(): void
+ menuLegajo(): void
+ menuEmpleado(): void
+ leerEntero(): int
+ leerEnteroLargo(): Long
+ crearLegajo(): void
+ mostrarLegajos(): void
+ leerLegajoPorId(): void
+ actualizarLegajo(): void
+ eliminarLegajo() : void
+ crearEmpleado() : void
+ leerEmpleadoPorId() : void
+ listarEmpleados() : void
+ buscarPorDni() : void
+ actualizarEmpleado() : void
+ eliminarEmpleado() : void



Empleado.java × Legajo.java ×

Source History

```java
package entities;

import java.time.LocalDate;

public class Empleado {
    private Long id;
    private Boolean eliminado = false;
    private String nombre;
    private String apellido;
    private String dni;
    private String email;
    private LocalDate fechaIngreso;
    private String area;
    private Legajo legajo; // Relacion 1-1 unidireccional

    // Constructores
    public Empleado() {}

    public Empleado(String nombre, String apellido, String dni, String email,
                    LocalDate fechaIngreso, String area, Legajo legajo) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.dni = dni;
        this.email = email;
        this.fechaIngreso = fechaIngreso;
        this.area = area;
        this.legajo = legajo;
    }

    // Getters y Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public Boolean getEliminado() { return eliminado; }
    public void setEliminado(Boolean eliminado) { this.eliminado = eliminado; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getApellido() { return apellido; }
    public void setApellido(String apellido) { this.apellido = apellido; }

    public String getDni() { return dni; }
    public void setDni(String dni) { this.dni = dni; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public LocalDate getFechaIngreso() { return fechaIngreso; }
    public void setFechaIngreso(LocalDate fechaIngreso) { this.fechaIngreso = fechaIngreso; }

    public String getArea() { return area; }
    public void setArea(String area) { this.area = area; }

    public Legajo getLegajo() { return legajo; }
    public void setLegajo(Legajo legajo) { this.legajo = legajo; }

    @Override
    public String toString() {
        return "Empleado{" +
                "id=" + id +
                ", nombre='" + nombre + ' ' + apellido + '\'' +
                ", dni='" + dni + '\'' +
                ", legajo=" + legajo +
                '}';
    }
```

Empleado.java ✕ Legajo.java ✕

Source History

```java
package entities;

import java.time.LocalDate;

public class Legajo {
    private Long id;
    private Boolean eliminado = false;
    private String nroLegajo;
    private String categoria;
    private Estado estado; // Enum: ACTIVO, INACTIVO
    private LocalDate fechaAlta;
    private String observaciones;

    public enum Estado {
        ACTIVO, INACTIVO
    }

    // Constructores
    public Legajo() {}

    public Legajo(String nroLegajo, String categoria, Estado estado, LocalDate fechaAlta, String observaciones) {
        this.nroLegajo = nroLegajo;
        this.categoria = categoria;
        this.estado = estado;
        this.fechaAlta = fechaAlta;
        this.observaciones = observaciones;
    }

    // Getters y Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public Boolean getEliminado() { return eliminado; }
    public void setEliminado(Boolean eliminado) { this.eliminado = eliminado; }

    public String getNroLegajo() { return nroLegajo; }
    public void setNroLegajo(String nroLegajo) { this.nroLegajo = nroLegajo; }

    public String getCategoria() { return categoria; }
    public void setCategoria(String categoria) { this.categoria = categoria; }

    public Estado getEstado() { return estado; }
    public void setEstado(Estado estado) { this.estado = estado; }

    public LocalDate getFechaAlta() { return fechaAlta; }
    public void setFechaAlta(LocalDate fechaAlta) { this.fechaAlta = fechaAlta; }

    public String getObservaciones() { return observaciones; }
    public void setObservaciones(String observaciones) { this.observaciones = observaciones; }

    @Override
    public String toString() {
        return "Legajo{" +
                "id=" + id +
                ", nroLegajo='" + nroLegajo + '\'' +
                ", estado=" + estado +
                ", fechaAlta=" + fechaAlta +
                '}';
    }
}
```

Diseño (UML):

Creé un diagrama UML de clases con todos los atributos (tipo y visibilidad) y métodos (firma y visibilidad).

Incluí una relación unidireccional explícita A -> B (1..1) entre las entidades: Empleado contiene un Legajo.

Definí los paquetes principales (entities, data, config) y las dependencias entre ellos.

El diagrama fue mi guía principal para escribir todo el código.

Entidades:

Ambas entidades, Empleado y Legajo, tienen sus atributos propios, un id (de tipo Long), y un campo eliminado (de tipo Boolean) para eliminación lógica.

Empleado contiene un atributo private Legajo legajo; (relación 1 a 1).

Cada entidad tiene un constructor vacío y uno completo, además de todos los getters/setters y un método toString() legible para mostrar sus datos.

---

2. Elección del Dominio y Justificación

Se seleccionó el dominio Empleado → Legajo debido a su alta aplicabilidad en entornos empresariales reales y su representatividad como caso de uso típico en sistemas de gestión de recursos humanos.

Esta relación 1→1 unidireccional refleja escenarios cotidianos donde cada empleado posee un único legajo institucional asociado, pero no es necesario que el legajo "conozca" al empleado que lo posee (unidireccionalidad). Esta elección permite modelar correctamente la integridad referencial y evitar duplicados, garantizando unicidad tanto en DNI como en número de legajo.

Además, este dominio facilita la implementación de validaciones importantes (como formatos de email, fechas válidas, estados de legajo) y permite simular operaciones transaccionales críticas: crear un empleado y su legajo en una misma transacción, asegurando consistencia ante fallos.

---

3. Diseño: Decisiones Clave y Diagrama UML

Relación 1→1 Unidireccional

La relación entre Empleado y Legajo es unidireccional 1→1, donde:

- La clase Empleado contiene un atributo privado: private Legajo legajo;.
- No existe referencia inversa desde Legajo hacia Empleado.

Estrategia de Persistencia: Clave Foránea Única

Optamos por la estrategia de clave foránea única en la tabla legajo:

Sql:

empleado_id BIGINT UNIQUE NOT NULL,

FOREIGN KEY (empleado_id) REFERENCES empleado(id) ON DELETE CASCADE

Justificación

- Garantiza que un Empleado solo pueda tener un Legajo.
- Permite fácil recuperación del legajo desde el empleado.
- Soporta eliminación en cascada si el empleado es borrado físicamente (aunque usamos baja lógica).
- Es más flexible que la clave primaria compartida, especialmente cuando Legajo puede evolucionar con más campos independientes.

Este diseño se refleja fielmente en el diagrama UML adjunto (uml.png), que incluye:

- Atributos y métodos con visibilidad.
- Tipos de datos correctos (Long, String, LocalDate, Enum, etc.).

- Relación 1→1 explícita desde Empleado a Legajo.
- Paquetes definidos: entities, dao, service, config, main.

---

4. Arquitectura por Capas

El proyecto sigue una arquitectura clara de capas, cumpliendo con buenas prácticas de diseño y separación de responsabilidades:

Capa	Responsabilidades
config	Gestión de la conexión a la base de datos mediante DatabaseConnection. Lectura de propiedades externas (futuro mejorable).
entities	Modelado de clases Empleado y Legajo con atributos, constructores, getters/setters y toString(). Incluyen id y eliminado para baja lógica.
dao	Implementación del patrón DAO usando JDBC y PreparedStatement. Operaciones CRUD básicas con soporte para Connection externa para transacciones.
service	Lógica de negocio: validaciones (campos obligatorios, formatos), reglas de dominio (unicidad de DNI, nroLegajo), y manejo transaccional (commit/rollback).
main	Punto de entrada de la aplicación. Contiene AppMenu con interfaz de consola interactiva para realizar todas las operaciones.

---

5. Persistencia: Estructura de la Base de Datos y Transacciones

Base de Datos

- Nombre: tfi_ddbb
- Motor: MySQL
- Puerto: 3306

Scripts SQL

Los scripts se ejecutan en MySQL Workbench:

- 01 - create_schema.sql: Crea la base de datos

- 02 - create_tables.sql: Crea todas las tablas con restricciones necesarias.
- 03 - Creación de índices.sql: Crea los índices de las tablas

Estructura de Tablas

Sql:

```
-- Creamos tabla Legajo con restricciones
CREATE TABLE IF NOT EXISTS Legajo (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    eliminado BOOLEAN NOT NULL DEFAULT FALSE, -- Baja lógica
    nroLegajo VARCHAR(20) NOT NULL UNIQUE, -- máx. 20
    categoria VARCHAR(30), -- máx. 30
    estado ENUM('ACTIVO', 'INACTIVO') NOT NULL, -- {ACTIVO, INACTIVO}
    fechaAlta DATE, -- java.time.LocalDate
    observaciones VARCHAR(255) -- máx. 255
);

-- Creamos tabla Empleado con restricciones y constraint
CREATE TABLE IF NOT EXISTS Empleado (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    eliminado BOOLEAN NOT NULL DEFAULT FALSE, -- Baja lógica
    nombre VARCHAR(80) NOT NULL, -- máx. 80
    apellido VARCHAR(80) NOT NULL, -- máx. 80
    dni VARCHAR(15) NOT NULL UNIQUE CHECK (length(dni) >= 8 and length(dni) < 16),
-- máx. 15
    email VARCHAR(120), -- máx. 120
    fechaIngreso DATE,
    area VARCHAR(50), -- máx. 50
    legajo BIGINT UNIQUE,
    CONSTRAINT fk_empleado_legajo
        FOREIGN KEY (legajo) REFERENCES Legajo(id)
                    ON UPDATE CASCADE
);
```

Manejo de Transacciones

En la capa Service:

1. Se desactiva autoCommit(false) al iniciar una operación compuesta.
2. Se realizan múltiples operaciones (ej: crear Empleado, luego crear Legajo y asociarlo).
3. Si todo sale bien → commit().
4. Si ocurre cualquier error → rollback() para mantener la consistencia.
5. Finalmente, se restaura autoCommit(true) y se cierran los recursos.

Esto asegura que no queden empleados sin legajo (o viceversa) en caso de fallo.

---

6. Validaciones y Reglas de Negocio

Las validaciones se implementan en la capa Service, antes de acceder a la base de datos:

- Campos obligatorios: nombre, apellido, dni, nroLegajo, estado, fechaAlta.
- Formatos:
  - Email: validación básica con expresión regular.
  - DNI: solo números y longitud razonable.
- Unicidad: Verificación previa antes de insertar para evitar violaciones de UNIQUE.
- Regla 1→1: Antes de asociar un nuevo Legajo a un Empleado, se verifica que no exista uno previo activo.
- Baja lógica: Los métodos eliminar() marcan eliminado = TRUE y no borran físicamente.

---

7. Pruebas Realizadas

Se realizaron pruebas manuales y simulaciones desde el menú de consola:

Escenarios Probados

- Creación exitosa de un Empleado con su Legajo asociado (transacción completa).
- Listado de empleados activos (filtrando por eliminado = FALSE).
- Búsqueda de empleado por DNI.
- Actualización de datos personales y estado del legajo.
- Eliminación lógica de un empleado (el legajo también queda marcado como eliminado).
- Simulación de error: intento de crear legajo con nroLegajo duplicado → se dispara rollback.

Capturas

Menú principal

```
Output - TrabajoPracticoIntegrador_Programacion_II (run)
run:
Usuario de MySQL: root
Contrase►a: 1234

Bases de datos disponibles:
1. gestionacademica
2. information_schema
3. libreria
4. mysql
5. performance_schema
6. sys
7. tpfi_ddbb
8. tup_tps
9. vuelos

Seleccione el n►mero de la base de datos: 7

MENU PRINCIPAL
1. Gestionar Legajo
2. Gestionar Empleado
0. Salir
Elija una opcion:
```

Opciones para Legajo

```
OPCIONES DISPONIBLES CON LEGAJO
1. Crear Legajo
2. Mostrar todos los legajos
3. Buscar por ID
4. Actualizar Legajo
5. Eliminar Legajo (marca logica)
0. Volver
Opcion:
```

Buscar empleado por ID

```
OPCIONES DISPONIBLES CON EMPLEADO
1. Crear Empleado
2. Mostrar todos los empleados activos
3. Buscar por ID
4. Buscar por DNI
5. Actualizar Empleado
6. Eliminar Empleado (marca logica)
0. Volver
Opcion: 3
ID del empleado: 1001
Empleado: Nombre1001 Apellido1001 | DNI: 00001001 | Area: DEV | Legajo: Nro: LEGAJO-00001001, Estado: ACTIVO, Categor►a: null
```

---

8. Conclusiones y Mejoras Futuras

Conclusiones

Este trabajo integrador permitió consolidar conocimientos clave en programación orientada a objetos, acceso a bases de datos con JDBC y diseño de arquitecturas por capas. Aprendimos a coordinar un proyecto complejo con múltiples componentes (entidades,

DAO, Service, transacciones), aplicando buenas prácticas como el manejo de excepciones, código limpio y separación de responsabilidades.

La implementación de una relación 1→1 unidireccional con persistencia real en MySQL reforzó la importancia del diseño previo (UML) y la coherencia entre modelo y base de datos.

Mejoras Futuras

- Agregar una interfaz gráfica (GUI) con JavaFX o Swing para mejorar la experiencia de usuario.
- Implementar tests unitarios con JUnit para validar servicios y DAOs.
- Incorporar logging (con SLF4J o Log4j) para registrar operaciones y errores.
- Permitir exportación de reportes (empleados activos, estadísticas de legajos).
- Usar archivos de propiedades para credenciales de base de datos, mejorando la portabilidad.

---

9. Fuentes y Herramientas Utilizadas

- Lenguaje: Java 21
- Base de Datos: MySQL 8.0
- IDE: Netbeans / IntelliJ IDEA / Eclipse (según integrante)
- Herramientas: MySQL Workbench, GitHub, Git
- Documentación: Oracle JDBC API, MySQL Docs
- IA Generativa: Se utilizó inteligencia artificial (IA) únicamente para ayudar en la redacción, formateo y estructuración del presente informe, asegurando claridad y profesionalismo en la entrega final. No se utilizó para generar lógica de negocio ni código funcional principal.

---

10. Enlace al Video:

https://drive.google.com/file/d/1rngVOi8TYAfpe1QRqil7M3AtdjlfZAZb/view?usp=drive_link

El video del equipo incluye:

- *Presentación de los integrantes.*
- *Demostración del menú de consola y flujos CRUD.*
- *Explicación técnica de las capas entities, dao, service y main.*
- *Simulación de una transacción con rollback ante error.*

Entregables incluidos en el repositorio GitHub:

- Código fuente completo
- README.md con instrucciones de instalación y ejecución
- schema.sql y data.sql
- Diagrama UML (uml.png)
- Informe en PDF
- Enlace al video